

Anatomia di un ciclo del browser – I

1. il client (browser) ottiene (come esito di un precedente transazione HTTP) una **pagina HTML**
2. il browser determina se la pagina HTML specifica ulteriori risorse necessarie ad una sua corretta rappresentazione grafica (detta anche **renderizzazione**).
3. non appena ha le risorse necessarie alla visualizzazione, il browser **renderizza** la pagina per mostrarla all'utente
4. non appena riceve gli eventuali script, il browser li esegue
5. **sopraggiunge un evento** (utente o di rete)
Se presente, il browser attiva la funzione precedentemente registrata per quell'evento. Tale funzione effettuerà elaborazioni, i cui effetti "percepibili" possono essere principalmente due: **generare altre connessioni HTTP** (per ottenere dei dati, o per spostarsi dalla pagina attuale), e **modificare l'aspetto della pagina** attualmente in visualizzazione.
6. quando l'aspetto della pagina viene modificato, il browser la **ri-renderizza**.

Interagire con DOM

Le interazioni tra i nostri script e il DOM ricadono in tre macrocategorie:

- **Navigazione o attraversamento del DOM.** In questa categoria rientrano tutte le operazioni volte a **trovare** un elemento o un insieme di elementi del DOM, sulla base della struttura dell'albero e/o del tipo/classe/id dell'elemento stesso, e a **leggerne** le caratteristiche rilevanti. Le operazioni di questa categoria vengono per lo più gestite tramite le proprietà e i metodi dell'interfaccia **Element**.
- **Modifica del DOM.** In questa categoria rientrano le operazioni volte ad aggiungere o rimuovere elementi del DOM, a spostare elementi da un punto all'altro dell'albero, a modificare le caratteristiche (ad es. le classi di appartenenza, o gli attributi in generale). Anche le operazioni di questa categoria vengono per lo più gestite tramite le proprietà e i metodi dell'interfaccia **Element**.
- **Gestione di eventi del DOM.** In questa categoria rientrano le operazioni volte a **registrare** (**subscribe/unsubscribe**) delle opportune funzioni come «ascoltatori» (**listener**) di degli eventi UI di interesse. La registrazione è specifica per una certa tipologia di evento e per l'elemento a partire dal quale l'evento si origina. I **listener** associati verranno invocati nel momento in cui quello specifico evento si verifica su quello specifico elemento: ciascun **listener** riceverà a quel punto le informazioni dettagliate sull'evento stesso. Le operazioni di questa categoria vengono gestite tramite le proprietà e i metodi dell'interfaccia **EventTarget**

In buona sostanza, alla base della programmazione di una UI in «vanilla Javascript» ci sono variazioni sul seguente loop: (1) trovare nel DOM gli elementi su cui intervenire, (2) registrare su di essi dei listener, (3) quando l'evento si verifica, modificare opportunamente il DOM.

Typescript

Carta di identità

Typescript aumenta la sintassi di Javascript con un **sistema di tipi**

In Typescript possiamo dichiarare il tipo atteso di variabili, parametri e valori di ritorno delle funzioni.

Typescript si **compila**, i programmi TS vengono tradotti in programmi JS

Dunque i costrutti «aggiuntivi» di TS sono **esclusivamente dichiarativi e non operazionali**: «scompaiono» effettuata la compilazione e non hanno nessun impatto sulla **esecuzione** del codice.

Typescript implementa un **sistema di inferenza dei tipi**

In fase di compilazione, TS determina il tipo risultante di funzioni ed espressioni, e verifica la consistenza con i tipi «promessi» come valori di ritorno, o con i modi in cui i risultati delle espressioni sono utilizzati.

Typescript è al suo meglio quando **integrato** in un **IDE**

L'inferenza sui tipi viene applicata già mentre scriviamo il codice, senza aspettare la compilazione. In questo modo è possibile prevenire molti errori di programmazione.

Web/HTTP Server

Principali funzioni

Fornire **contenuti statici** e **dinamici** tramite protocollo HTTP

Gestione di **host virtuali**

Autorizzazione e **autenticazione**

Caching di richieste e relative risposte

Riduzione di banda (**bandwith throttling**)

Riscrittura delle URL (**rewrite engine**)

Logging di informazioni sugli scambi HTTP per scopi statistici e di security

JPA

Ciclo di vita delle entità - 1

Quando un'entità viene creata in memoria, è **nuova** (new), JPA non la gestisce e il DB non ha idea che esista.

Quando un'entità viene salvata tramite **repo.save(e)** viene effettuata una **create**, e l'entità diventa **managed**, ossia gestita.
Lo resta finché non termina la **transazione** corrente.

Anche quando estraiamo un'entità dal DB tramite uno dei metodi del **repository** (per esempio, **findById**) l'entità è **managed** fino al termine della **transazione**.

Al termine della **transazione** corrente, l'entità diventa **detached**, «staccata».

Se voglio riagganciare un'entità **detached**, posso chiamare di nuovo **save**. Poiché l'entità ha un id presente nel DB, la **save** effettuerà **update** (anziché **create**)

Quando cancelliamo un'entità con **repo.delete** l'entità diventa **removed**: non appena la **transazione** termina, essa scomparirà dal DB

Model-View-ViewModel

Pattern architetturale per UI a componenti

Modello dati:

i dati che l'app elabora.
i vincoli che ne regolano
la fruizione e l'elaborazione

View-Model:

il **comportamento**
dei componenti

Componenti in React: il ViewModel del MVVM

View:

ciò che l'utente vede
e con cui interagisce
(il DOM)

